- *Concurrency*: Sidekiq needs to know how many threads to run per server process. You are likely to want to change this without having to wait for a re-deploy so allowing an environment variable to override a default is a good strategy here.

- *Timeout*: When Sidekiq asks a job to terminate, the timeout is how long it will wait for forcing a job to stop. This is another value you may want to change without a re-deploy, so you can use an environment variable that overrides a default.

- *Queues*: Queues aren't added often, and they must be accompanied by a code change that uses the queue. Explicitly hard-code the list of queues to Sidekiq's default value so you know where to change it in the future.

To connect to Redis, you'll use a few environment variables. In the example app's development and test environment, the environment variables are managed by the dotenv gem.[5] That gem will examine the files `.env.development` and `.env.test` and use them to set environment variables for development and testing, respectively.

If you look at the files in the example app, you can see that there is a Redis URL set to the environment variable `SIDEDKIQ_REDIS_URL` (this is already set so that you could validate the example app's ability to access Redis before installing Sidekiq):

snapshots/0-0/sidekiq-book/.env.development

```
DATABASE_URL=postgres://postgres:postgres@db:5432/sidekiq-book_development
SIDEKIQ_REDIS_URL=redis://redis:6379/1
FULLFILLMENT_API_URL=http://fake-api-server:4000/fulfillment
PAYMENTS_API_URL=http://fake-api-server:4000/payments
EMAIL_API_URL=http://fake-api-server:4000/email
ERROR_CATCHER_API_URL=http://fake-api-server:4000/error-catcher

# Limit all requests to 5 seconds
RACK_TIMEOUT_SERVICE_TIMEOUT=5
```